



Bikkad IT Institute

NumPy Tricks Notes

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

NumPy Tricks Notes

Sorting, Searching, Set Ops and Useful Functions

Prepared for Bikkad IT Institute Students | Level: Beginner to Intermediate | Environment:
Jupyter Notebook or Google Colab

Topic Index

- 1. np.sort
- 2. np.append
- 3. np.concatenate
- 4. np.unique
- 5. np.expand_dims
- 6. np.where
- 7. np.argmax and np.argmin
- 8. np.cumsum and np.cumprod
- 9. np.percentile and np.median
- 10. np.histogram
- 11. np.corrcoef
- 12. np.isin
- 13. np.flip
- 14. np.put and np.delete
- 15. Set functions
- 16. np.clip

1. np.sort

Returns a sorted copy of an array, without changing the original.

Syntax

```
np.sort(a, axis=-1)
```

OUR LOCATIONS

Example 1: Ascending sort of a 1D array

```
import numpy as np
np.random.seed(1)
a = np.random.randint(1, 100, 8)
print('Original:', a)
print('Sorted :', np.sort(a))
```

Output / Result

```
Original: [38 13 73 10 76 6 80 65]
Sorted : [ 6 10 13 38 65 73 76 80]
```

Explanation

`np.sort()` returns a new array with the values arranged from smallest to largest by default; the original array `a` is left unchanged.

Example 2: Descending sort

```
print(np.sort(a)[::-1])
```

Output / Result

```
[80 76 73 65 38 13 10 6]
```

Explanation

NumPy has no direct 'descending' argument, so the usual trick is to sort ascending and then reverse the result with `[::-1]`.

Example 3: Sorting a 2D array along an axis

```
b = np.random.randint(1, 100, 12).reshape(3, 4)
print(b)
print(np.sort(b, axis=0))
```

Output / Result

```
[[17 2 77 72]
 [ 7 26 51 21]
 [19 85 12 29]]
[[ 7 2 12 21]
 [17 26 51 29]
 [19 85 77 72]]
```

Explanation

`axis=0` sorts each column independently (down the rows); `axis=1` (the default for 2D arrays) would sort each row independently instead.

OUR LOCATIONS

2. np.append

Appends values along the given axis at the end of an array, returning a new array.

Syntax

```
np.append(array, values, axis=None)
```

Example 1: Appending a single value

```
a = np.array([10, 20, 30])
print(np.append(a, 200))
```

Output / Result

```
[ 10 20 30 200]
```

Explanation

Without an axis, the array is treated as flat and the new value is simply added at the end.

Example 2: Appending a new column to a 2D array

```
np.random.seed(2)
b = np.arange(8).reshape(4, 2)
extra_col = np.random.randint(1, 10, (b.shape[0], 1))
print(b)
print(np.append(b, extra_col, axis=1))
```

Output / Result

```
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
[[0 1 9]
 [2 3 9]
 [4 5 7]
 [6 7 3]]
```

Explanation

axis=1 appends the new values as an additional column, so the appended array must have the same number of rows as b.

OUR LOCATIONS

Example 3: Appending a new row to a 2D array

```
new_row = np.array([[99, 99]])  
print(np.append(b, new_row, axis=0))
```

Output / Result

```
[[ 0 1]  
 [ 2 3]  
 [ 4 5]  
 [ 6 7]  
 [99 99]]
```

Explanation

axis=0 appends the new values as an additional row, so the appended array must have the same number of columns as b.

3. np.concatenate

Joins a sequence of arrays along an existing axis.

Syntax

```
np.concatenate((a, b), axis=0)
```

Example 1: Concatenating along axis 0 (rows)

```
c = np.arange(6).reshape(2, 3)  
d = np.arange(6, 12).reshape(2, 3)  
print(c)  
print(d)  
print(np.concatenate((c, d), axis=0))
```

Output / Result

```
[[0 1 2]  
 [3 4 5]]  
[[ 6 7 8]  
 [ 9 10 11]]  
[[ 0 1 2]  
 [ 3 4 5]  
 [ 6 7 8]  
 [ 9 10 11]]
```

Explanation

axis=0 stacks the arrays vertically, one below the other; both arrays must have the same number of columns.

OUR LOCATIONS

Example 2: Concatenating along axis 1 (columns)

```
print(np.concatenate((c, d), axis=1))
```

Output / Result

```
[[ 0  1  2  6  7  8]
 [ 3  4  5  9 10 11]]
```

Explanation

axis=1 joins the arrays side by side; both arrays must have the same number of rows.

Example 3: Concatenating flat 1D arrays

```
e1 = np.array([1, 2, 3])
e2 = np.array([4, 5])
print(np.concatenate((e1, e2)))
```

Output / Result

```
[1 2 3 4 5]
```

Explanation

For 1D arrays there is only one axis, so concatenate simply chains the values end to end.

4. np.unique

Finds the distinct (unique) values in an array, sorted in ascending order.

Syntax

```
np.unique(a, return_counts=False, return_index=False)
```

Example 1: Basic unique values

```
arr = np.array([1, 1, 2, 2, 3, 3, 4, 4, 5])
print(np.unique(arr))
```

Output / Result

```
[1 2 3 4 5]
```

Explanation

Duplicate values are collapsed and the remaining distinct values are returned in sorted order.

OUR LOCATIONS

Example 2: Counting how often each value occurs

```
values, counts = np.unique(arr, return_counts=True)
print(values)
print(counts)
```

Output / Result

```
[1 2 3 4 5]
[2 2 2 2 1]
```

Explanation

`return_counts=True` additionally returns how many times each unique value appeared in the original array.

Example 3: Locating the first occurrence of each value

```
values, idx = np.unique(arr, return_index=True)
print(values)
print(idx)
```

Output / Result

```
[1 2 3 4 5]
[0 2 4 6 8]
```

Explanation

`return_index=True` returns the position in the original array where each unique value first appears.

5. np.expand_dims

Inserts a new axis of size 1 at the given position, increasing an array's dimension count.

Syntax

```
np.expand_dims(a, axis)
```

Example 1: Expanding a 1D array along axis 0

```
a = np.array([1, 2, 3, 4])
print('Original shape:', a.shape)
print('New shape :', np.expand_dims(a, axis=0).shape)
```

Output / Result

```
Original shape: (4,)
New shape : (1, 4)
```

Explanation

`axis=0` turns a flat (4,) array into a (1,4) row vector by adding a new leading dimension.

OUR LOCATIONS

Example 2: Expanding along axis 1

```
expanded = np.expand_dims(a, axis=1)
print(expanded)
print('New shape:', expanded.shape)
```

Output / Result

```
[[1]
 [2]
 [3]
 [4]]
New shape: (4, 1)
```

Explanation

axis=1 instead turns the same array into a (4,1) column vector, adding the new dimension at the end.

Example 3: Comparing both against the original

```
print('a.shape :', a.shape)
print('expand_dims(a,0):', np.expand_dims(a, 0).shape)
print('expand_dims(a,1):', np.expand_dims(a, 1).shape)
```

Output / Result

```
a.shape : (4,)
expand_dims(a,0): (1, 4)
expand_dims(a,1): (4, 1)
```

Explanation

expand_dims is commonly used to reshape a 1D array into a row or column vector before a broadcasting operation that needs an explicit 2D shape.

6. np.where

Returns the indices where a condition holds, or builds a new array by choosing between two values based on that condition.

Syntax

```
np.where(condition)
np.where(condition, value_if_true, value_if_false)
```

OUR LOCATIONS

Example 1: Finding indices where a condition holds

```
np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)
print(np.where(a > 50))
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
(array([2, 3, 7]),)
```

Explanation

With a single argument, `np.where(condition)` returns a tuple of index arrays for every position where the condition is True.

Example 2: Replacing values above a threshold

```
print(np.where(a > 50, 0, a))
```

Output / Result

```
[25  4  0  0  1 22 20  0 42 11]
```

Explanation

With three arguments, `np.where` rebuilds the array: wherever the condition is True it uses the second argument (0), otherwise it keeps the original value.

Example 3: Replacing every even value

```
print(np.where(a % 2 == 0, 0, a))
```

Output / Result

```
[25  0 57 73  1  0  0 75  0 11]
```

Explanation

The condition can be any Boolean expression - here every even number is replaced with 0 while odd numbers are left untouched.

7. np.argmax and np.argmin

Return the index (position) of the maximum or minimum element, rather than the value itself.

Syntax

```
np.argmax(a, axis=None)
np.argmin(a, axis=None)
```

OUR LOCATIONS

Example 1: Position of the largest and smallest value

```
np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)
print('argmax:', np.argmax(a))
print('argmin:', np.argmin(a))
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
argmax: 7
argmin: 4
```

Explanation

argmax and argmin return the position of the extreme value, which is often more useful than the value itself when you need to know where it occurred.

Example 2: argmax along columns of a 2D array

```
b = np.random.randint(1, 100, 12).reshape(3, 4)
print(b)
print(np.argmax(b, axis=0))
```

Output / Result

```
[[22 39 97 21]
 [45 94 40 15]
 [27 82 91 23]]
[1 1 0 2]
```

Explanation

axis=0 returns, for each column, the row index of that column's largest value.

Example 3: argmax along rows of a 2D array

```
print(np.argmax(b, axis=1))
```

Output / Result

```
[2 1 2]
```

Explanation

axis=1 instead returns, for each row, the column index of that row's largest value.

8. np.cumsum and np.cumprod

Compute a running (cumulative) sum or product of array elements, rather than a single total.

Syntax

```
np.cumsum(a, axis=None)
np.cumprod(a, axis=None)
```

Example 1: Cumulative sum of a 1D array

```
a = np.array([5, 10, 15, 20])
print(np.cumsum(a))
```

Output / Result
[5 15 30 50]

Explanation

Each output position holds the running total of all elements up to and including that position.

Example 2: Cumulative sum along an axis of a 2D array

```
b = np.arange(1, 13).reshape(3, 4)
print(b)
print(np.cumsum(b, axis=1))
```

Output / Result
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]]
[[1 3 6 10]
 [5 11 18 26]
 [9 19 30 42]]

Explanation

axis=1 restarts the running total at the beginning of each row; omitting axis would flatten the array first and cumulate over all elements.

Example 3: Cumulative product

```
print(np.cumprod(a))
```

Output / Result
[5 50 750 15000]

Explanation

np.cumprod works exactly like cumsum but multiplies the running total instead of adding to it.

OUR LOCATIONS

9. np.percentile and np.median

Summarize the relative position of values in a dataset; the median is simply the 50th percentile.

Syntax

```
np.percentile(a, q)
np.median(a)
```

Example 1: The 50th percentile

```
np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)
print(np.percentile(a, 50))
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
23.5
```

Explanation

The 50th percentile is the value below which half the data falls, exactly matching the definition of the median.

Example 2: Confirming it matches np.median

```
print(np.median(a))
```

Output / Result

```
23.5
```

Explanation

`np.median(a)` is a convenience shortcut that always returns `np.percentile(a, 50)`.

Example 3: Multiple percentiles at once (quartiles)

```
print(np.percentile(a, [25, 50, 75]))
```

Output / Result

```
[13.25 23.5 53.25]
```

Explanation

Passing a list of percentages computes several percentiles in a single call, commonly used to get the quartiles of a dataset.

10. np.histogram

Counts how many values fall into each of a set of bins, the numeric basis behind a histogram chart.

Syntax

```
np.histogram(a, bins)
```

Example 1: Two wide bins

```
np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)
counts, edges = np.histogram(a, bins=[0, 50, 100])
print('Counts:', counts)
print('Edges :', edges)
```

Output / Result

```
[25 4 57 73 1 22 20 75 42 11]
Counts: [7 3]
Edges : [ 0 50 100]
```

Explanation

The bins list defines the boundaries; counts reports how many values fall inside each interval, here below and above 50.

Example 2: More, evenly-sized bins

```
counts, edges = np.histogram(a, bins=[0, 25, 50, 75, 100])
print('Counts:', counts)
print('Edges :', edges)
```

Output / Result

```
Counts: [5 2 2 1]
Edges : [ 0 25 50 75 100]
```

Explanation

More boundary points create a finer-grained frequency breakdown, which is exactly what a histogram plot visualizes as bars.

OUR LOCATIONS

Example 3: Automatic equal-width bins

```
counts, edges = np.histogram(a, bins=4)
print('Counts:', counts)
print('Edges :', np.round(edges, 1))
```

Output / Result

```
Counts: [3 3 1 3]
Edges : [ 1. 19.5 38. 56.5 75. ]
```

Explanation

Passing a single integer instead of a list asks NumPy to divide the data's range into that many equal-width bins automatically.

11. np.corrcoef

Returns the Pearson correlation coefficient matrix, measuring how strongly two variables move together.

Syntax

```
np.corrcoef(x, y)
```

Example 1: A strong positive correlation

```
salary = np.array([20000, 30000, 40000, 50000, 60000])
experience = np.array([1, 2, 3, 5, 4])
print(np.round(np.corrcoef(salary, experience), 3))
```

Output / Result

```
[[1. 0.9]
 [0.9 1. ]]
```

Explanation

The off-diagonal value shows the correlation between salary and experience; a value close to 1 means they tend to increase together.

OUR LOCATIONS

Example 2: A near-zero correlation

```
np.random.seed(4)
x = np.arange(10)
y = np.random.randint(1, 100, 10)
print(np.round(np.corrcoef(x, y), 3))
```

Output / Result

```
[[1. 0.174]
 [0.174 1. ]]
```

Explanation

When one variable is essentially unrelated to the other, the off-diagonal correlation value sits close to 0.

Example 3: Every array perfectly correlates with itself

```
print(np.corrcoef(salary, salary))
```

Output / Result

```
[[1. 1.]
 [1. 1.]]
```

Explanation

The diagonal of any correlation matrix is always 1, since a variable is always perfectly correlated with itself.

12. np.isin

Checks, element by element, whether values from one array also appear in another array.

Syntax

```
np.isin(a, test_values)
```

Example 1: Keeping only values from a target list

```
a = np.array([15, 20, 35, 40, 55, 60, 75, 80])
items = [20, 40, 60, 80, 100]
print(a[np.isin(a, items)])
```

Output / Result

```
[20 40 60 80]
```

Explanation

`np.isin` builds a Boolean mask marking every element of `a` that also occurs in `items`, which can then be used to filter `a` directly.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 2: Excluding values with the negation

```
print(a[~np.isin(a, items)])
```

Output / Result

```
[15 35 55 75]
```

Explanation

Prefixing the mask with ~ flips True and False, so this keeps every element that is NOT found in items.

Example 3: Using isin on a 2D array

```
b = np.arange(12).reshape(3, 4)
print(b)
print(np.isin(b, [0, 5, 10]))
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ True False False False]
 [False True False False]
 [False False True False]]
```

Explanation

isin works on arrays of any shape; the result is a same-shaped Boolean array rather than a flat list.

13. np.flip

Reverses the order of array elements along the given axis while preserving the array's shape.

Syntax

```
np.flip(a, axis=None)
```

Example 1: Flipping a 1D array

```
a = np.array([10, 20, 30, 40, 50])
print(np.flip(a))
```

Output / Result

```
[50 40 30 20 10]
```

Explanation

Without an axis, flip reverses the entire flattened sequence of elements, similar to a[::-1].

OUR LOCATIONS

Example 2: Flipping a 2D array along columns

```
b = np.arange(12).reshape(3, 4)
print(b)
print(np.flip(b, axis=1))
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 3  2  1  0]
 [ 7  6  5  4]
 [11 10  9  8]]
```

Explanation

axis=1 reverses the order of columns within every row, while the row order itself stays the same.

Example 3: Flipping along rows instead

```
print(np.flip(b, axis=0))
```

Output / Result

```
[[ 8  9 10 11]
 [ 4  5  6  7]
 [ 0  1  2  3]]
```

Explanation

axis=0 instead reverses the order of the rows, while each row's own contents stay in the same order.

14. np.put and np.delete

np.put replaces specific elements of an array in place; np.delete returns a new array with certain elements removed.

Syntax

```
np.put(a, indices, values)
np.delete(a, indices, axis=None)
```

OUR LOCATIONS

Example 1: Replacing elements in place with np.put

```
a = np.array([10, 20, 30, 40, 50])
np.put(a, [0, 1], [110, 530])
print(a)
```

Output / Result

```
[110 530 30 40 50]
```

Explanation

np.put treats the array as flattened and directly overwrites the values at the given positions - unlike most NumPy functions, it modifies a in place rather than returning a new array.

Example 2: Deleting elements from a 1D array

```
b = np.array([10, 20, 30, 40, 50])
print(np.delete(b, [0, 2, 4]))
```

Output / Result

```
[20 40]
```

Explanation

np.delete returns a new array with the elements at the given positions removed, leaving the original array b untouched.

Example 3: Deleting a whole row from a 2D array

```
c = np.arange(12).reshape(3, 4)
print(c)
print(np.delete(c, 1, axis=0))
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2  3]
 [ 8  9 10 11]]
```

Explanation

Passing axis=0 removes an entire row (here, row index 1) instead of individual flattened elements.

OUR LOCATIONS

15. Set functions

NumPy treats 1D arrays as mathematical sets, offering union, intersection, difference, and membership operations.

Syntax

```
np.union1d(a, b); np.intersect1d(a, b)
np.setdiff1d(a, b); np.setxor1d(a, b)
```

Example 1: Union and intersection

```
m = np.array([1, 2, 3, 4, 5])
n = np.array([3, 4, 5, 6, 7])
print('Union :', np.union1d(m, n))
print('Intersection:', np.intersect1d(m, n))
```

Output / Result

```
Union : [1 2 3 4 5 6 7]
Intersection: [3 4 5]
```

Explanation

Union keeps every distinct value from both arrays; intersection keeps only the values common to both.

Example 2: Difference and symmetric difference

```
print('n - m :', np.setdiff1d(n, m))
print('Sym. diff :', np.setxor1d(m, n))
```

Output / Result

```
n - m : [6 7]
Sym. diff : [1 2 6 7]
```

Explanation

setdiff1d(n, m) keeps values in n that are not in m; setxor1d keeps values present in exactly one of the two arrays.

Example 3: Checking membership with in1d

```
print(m[np.isin(m, [1])])
```

Output / Result

```
[1]
```

Explanation

Filtering with a membership test (the modern equivalent of the older np.in1d) picks out only the elements of m that match a given value or list of values.

OUR LOCATIONS

16. np.clip

Limits (clips) every value in an array to stay within a given minimum and maximum range.

Syntax

```
np.clip(a, a_min, a_max)
```

Example 1: Clipping to both a minimum and maximum

```
np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)
print(np.clip(a, a_min=25, a_max=75))
```

Output / Result

```
[25 4 57 73 1 22 20 75 42 11]
[25 25 57 73 25 25 25 75 42 25]
```

Explanation

Every value below 25 is raised to 25, every value above 75 is lowered to 75, and values already inside the range are left unchanged.

Example 2: Clipping with only a lower bound

```
print(np.clip(a, a_min=50, a_max=None))
```

Output / Result

```
[50 50 57 73 50 50 50 75 50 50]
```

Explanation

Passing `a_max=None` leaves the upper end unrestricted, so this only raises values below 50 up to 50.

Example 3: Clipping with only an upper bound

```
print(np.clip(a, a_min=None, a_max=50))
```

Output / Result

```
[25 4 50 50 1 22 20 50 42 11]
```

Explanation

Passing `a_min=None` leaves the lower end unrestricted, so this only lowers values above 50 down to 50.